

LABORATORY OF AUTOMATION SYSTEMS

Davide Samorì

Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione (DEI)

Università di Bologna

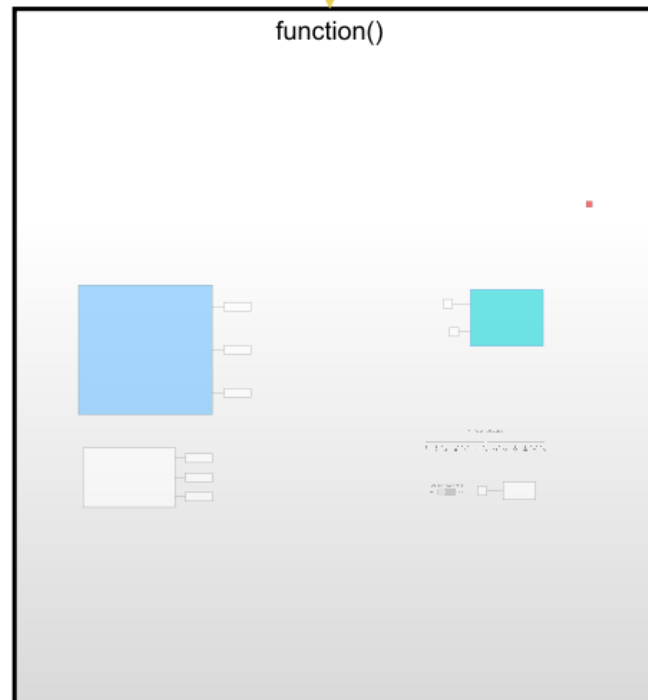
Email: davide.samori@unibo.it

You can download and open:

- Your DC Motor Model
- Slide **LAS_03_Motion.pdf** (Prof. Melchiorri)

Today Goals:

- PI Design
- Current Loop
- Speed Loop
- Saturation Problem

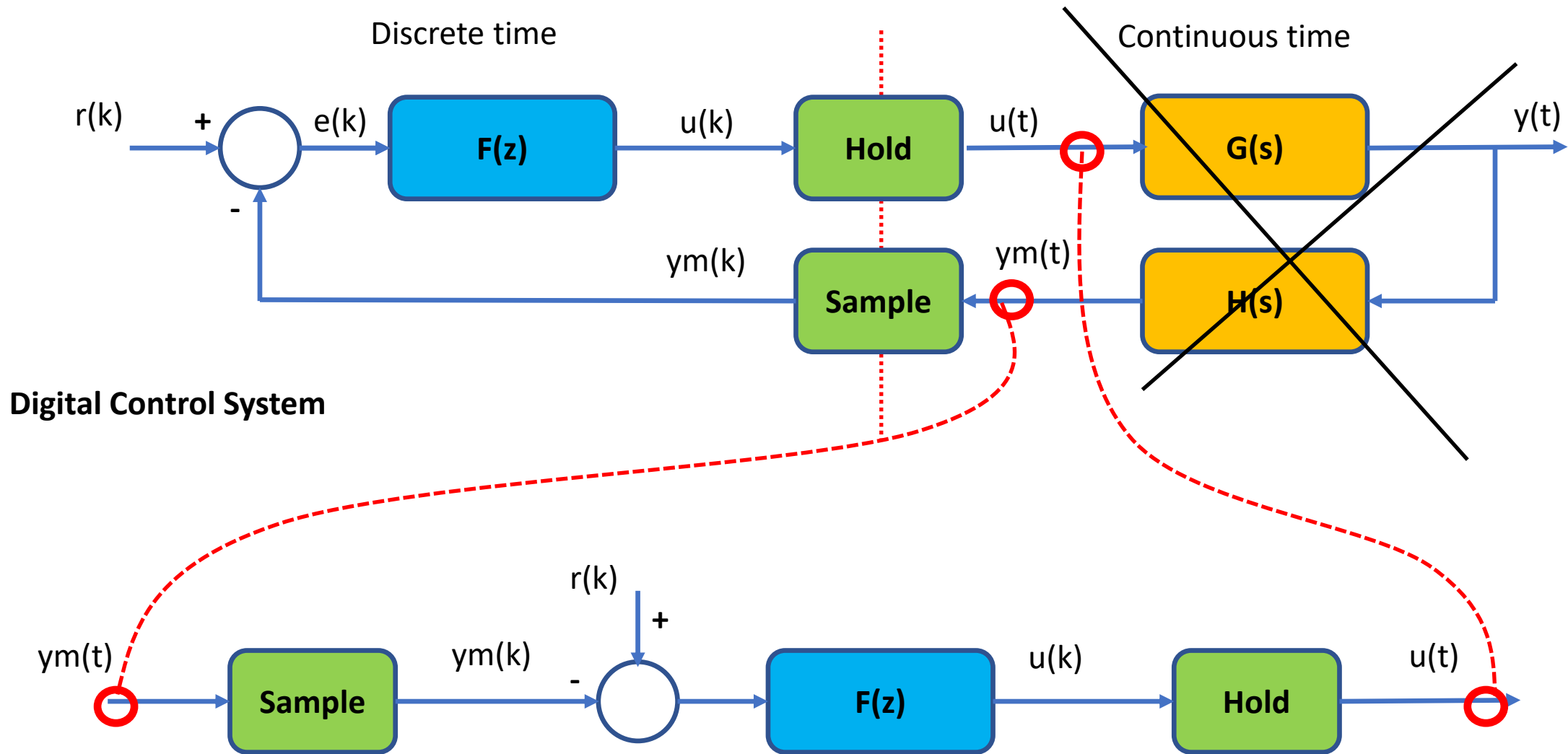


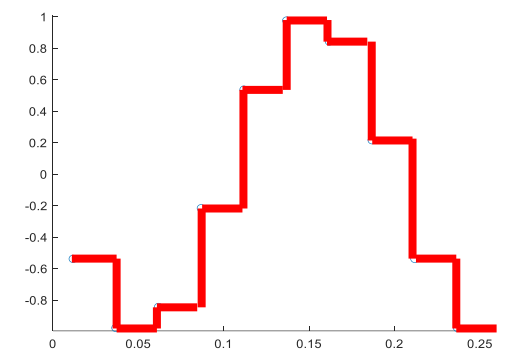
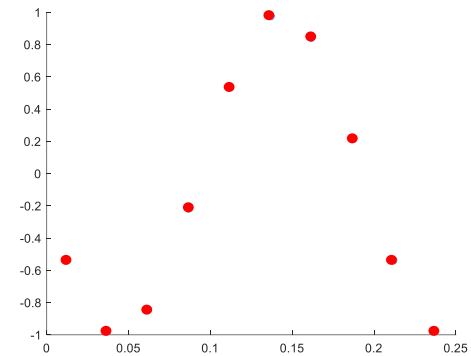
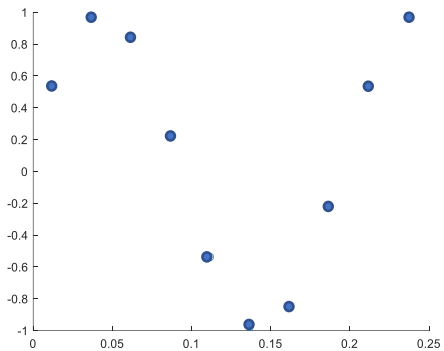
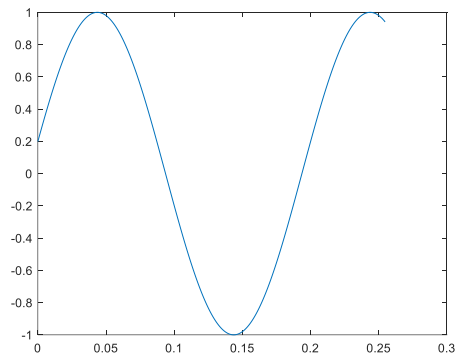
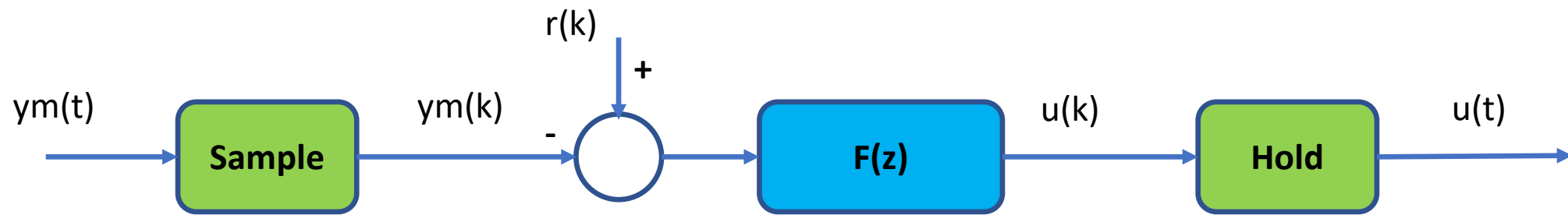
This is our main task.
Every TSample=100us we run it

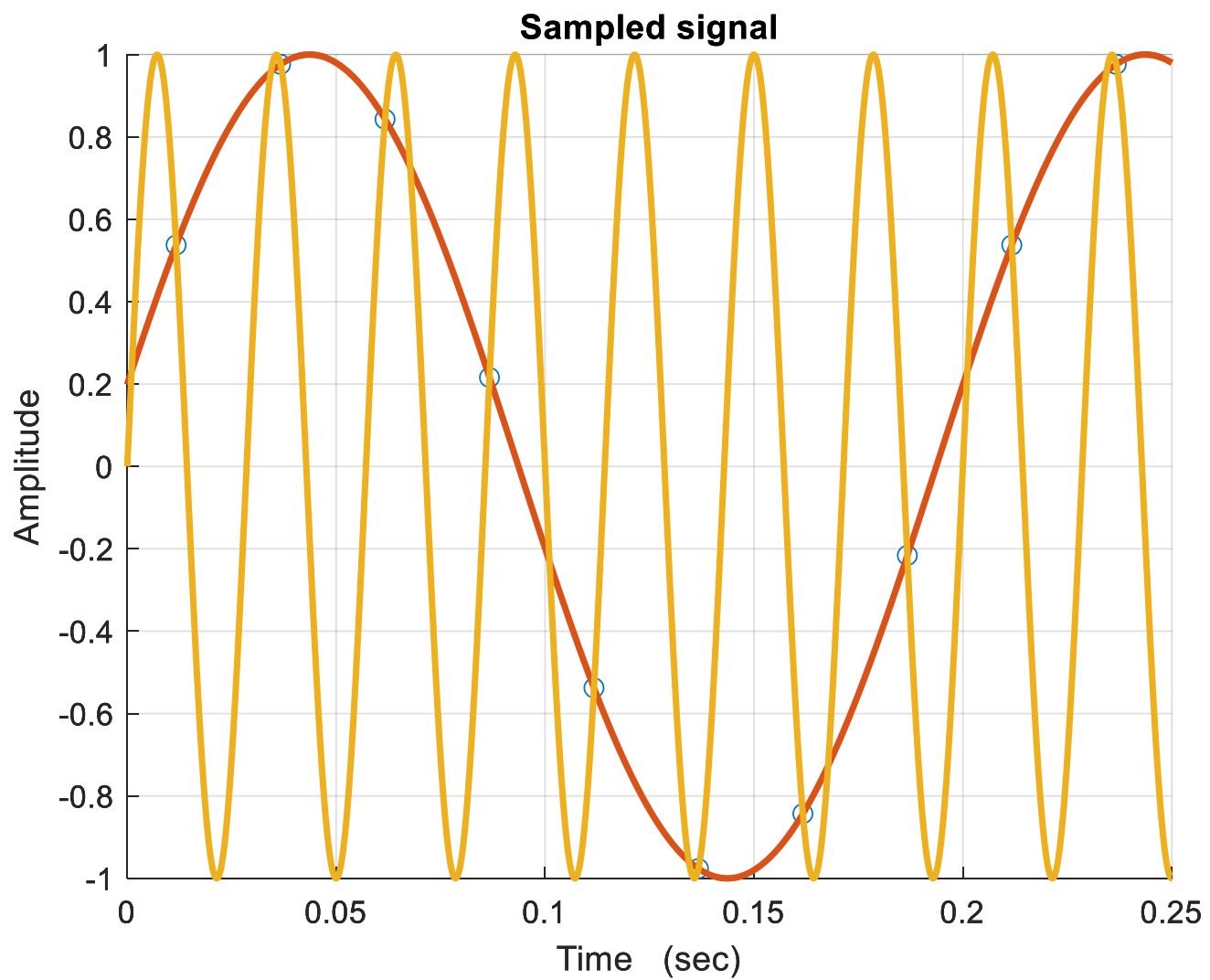


Something about sample time...

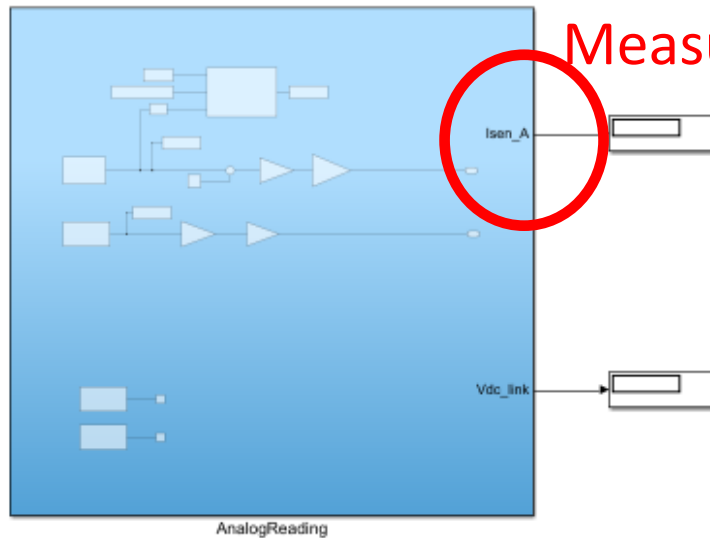
T=tempo di campionamento





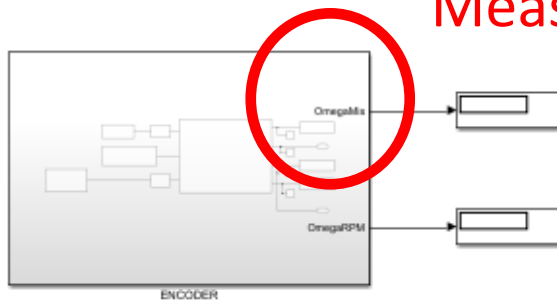
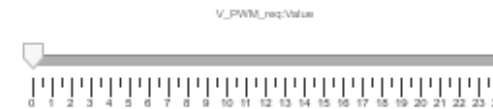
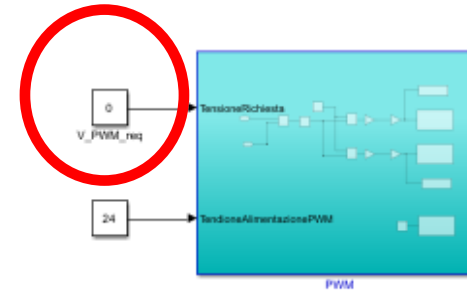


- How to choose sample time:
 - Shannon
 - PWM
 - Dynamics
 - Execution Time
 -



Measured Current

Voltage to the motor



Measure speed (rad/s)

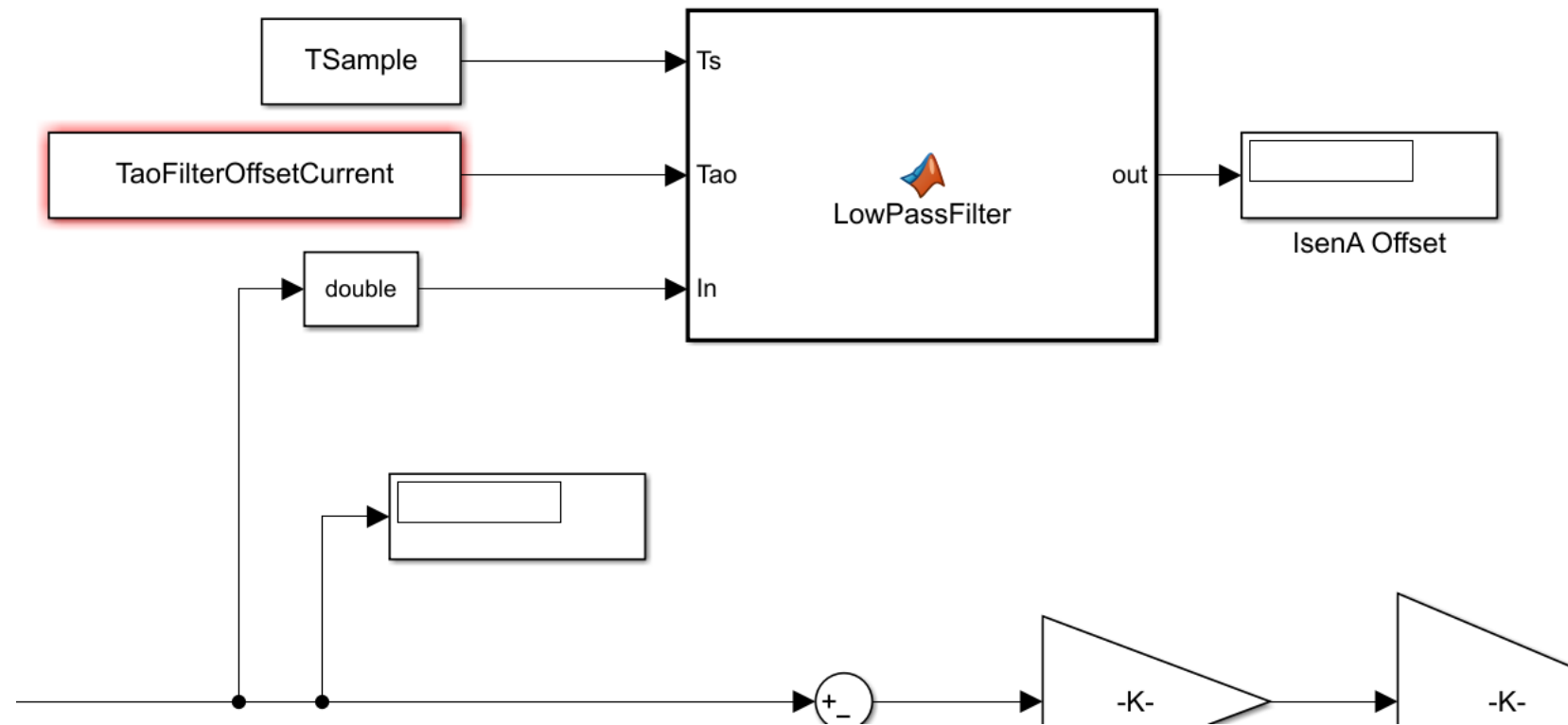


We don't have motor model, we use real motor



A quick review about the filter

We realized a discrete time filter



EXAMPLE: FILTER BACKWORD DIFFERENCE DISCRETIZATION

$$G(s) = \frac{1}{1 + s\tau} \quad \left| \quad s = \frac{z - 1}{Tz} = \frac{1 - z^{-1}}{T} \right.$$

$$G(z) = \frac{1}{1 + \frac{1 - z^{-1}}{T}\tau} = \frac{Y}{X} \quad \longrightarrow \quad Y \left(1 + \frac{\tau}{T} - \frac{\tau}{T} z^{-1} \right) = X \quad \longrightarrow \quad Y \left(1 + \frac{\tau}{T} \right) = X + Y \frac{\tau}{T} z^{-1}$$

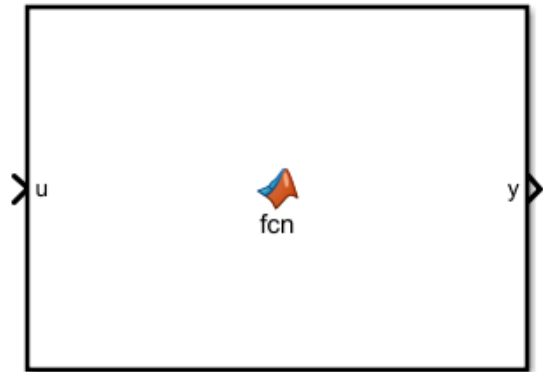
$$Y = \frac{T}{T + \tau} X + \frac{\tau}{T + \tau} Y z^{-1}$$

$\longrightarrow$

$$Y = \frac{T}{T + \tau} X + \frac{\tau}{T + \tau} Y_{\text{prec}}$$

T=tempo di campionamento

Matlab Function



Doppio click

```
function y = fcn(u)
```

```
y = u;
```

Prendiamo una Matlab Function



$$Y = \frac{T}{T+\tau} X + \frac{\tau}{T+\tau} Y_{\text{prec}}$$

```
function out = LowPassFilter( Ts ,Tao, In)

persistent OutPrev
if isempty(OutPrev )
    OutPrev = 0 ;
end

% Low-Pass filter
% discretized by backward difference

a=Ts/ (Ts+Tao) ;
b=Tao/ (Ts+Tao) .

out=a*In + OutPrev *b;

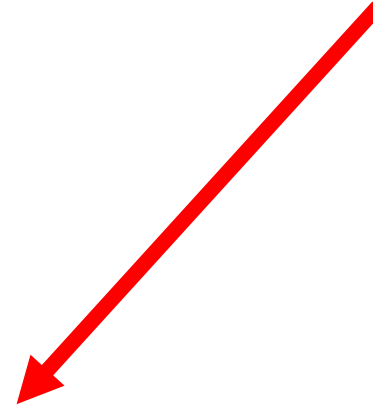
OutPrev = out;

end
```

$$G(s) = \frac{1}{1 + s\tau} \quad \left| \quad s = \frac{1 - z^{-1}}{T} \right.$$



$$Y = \frac{T}{T + \tau} X + \frac{\tau}{T + \tau} Y_{\text{prec}}$$



T=tempo di campionamento



PI !!

PI: TUSTIN DISCRETIZATION

YOU CAN TRY WITH DIFFERENT DISCRETIZATIN TECNIQUES

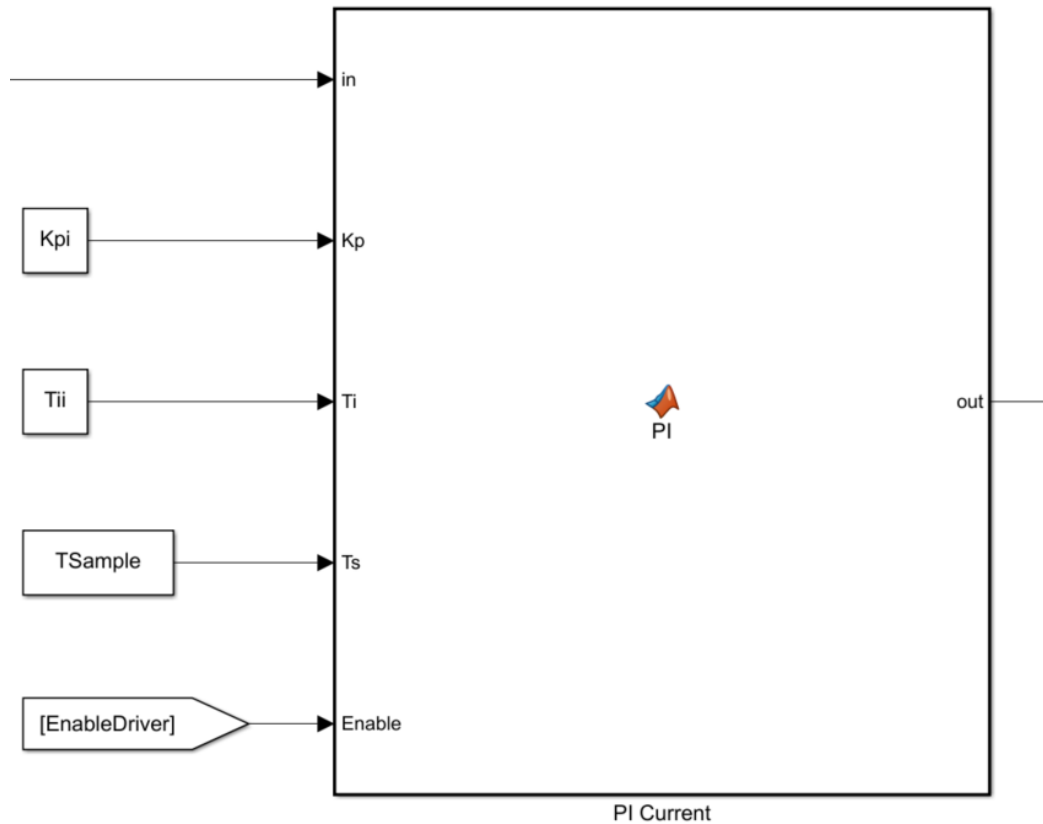
$$R(s) = K_p \cdot \left(1 + \frac{1}{T_i \cdot s}\right) \quad \left| \quad s = \frac{2}{T_s} \frac{1 - z^{-1}}{1 + z^{-1}} \right.$$

$$R(z) = K_p \cdot \left(1 + \frac{1}{T_i \cdot \frac{2}{T_s} \frac{1 - z^{-1}}{1 + z^{-1}}}\right) = \frac{Y}{X} \quad \longrightarrow \quad \dots \dots \dots$$



$$Y = K_p \cdot (X - X_{\text{prec}}) + \frac{K_p \cdot T_s}{2 \cdot T_i} \cdot (X + X_{\text{prec}}) + Y_{\text{prec}}$$

Ts=tempo di campionamento



```
function out = PI(in ,Kp , Ti , Ts , Enable)
```

```
persistent in_prev out_prev
```

```
% Initial Value
```

```
if isempty( out_prev )
    out_prev = 0;
end
```

```
end
```

```
if isempty( in_prev )
    in_prev = 0;
end
```

```
end
```

```
%  $G(s) = (Kp/Ti) * ((1+Ti*s)/s)$  (Tustin)
```

```
% We run control only if motor is running
if (Enable == 1) ←
```

```
    %I Calculate the output
```

```
    out = Kp*Ts/(2*Ti)*(in+in_prev) + Kp*(in-in_prev) + out_prev;
```

```
else
```

```
    out = 0;
```

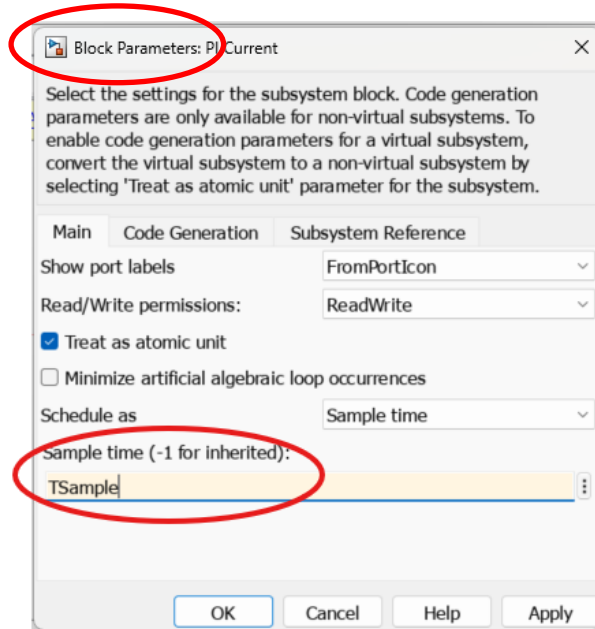
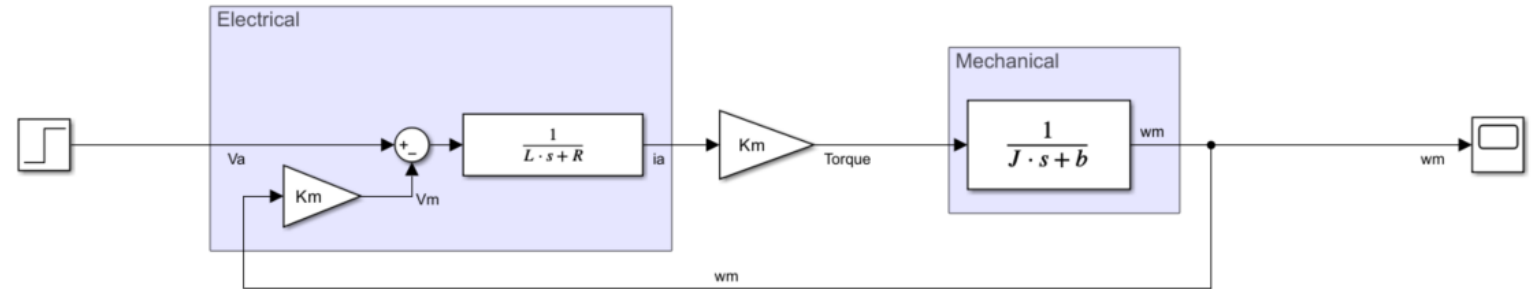
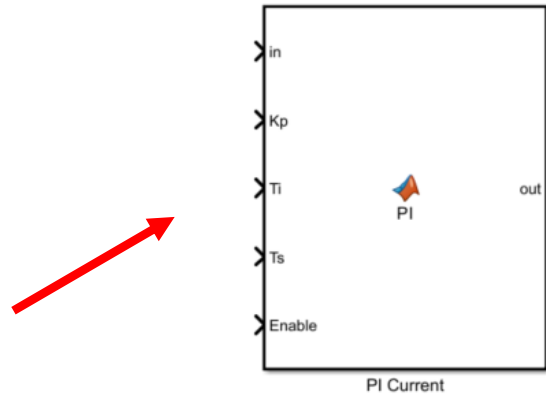
```
    in = 0;
```

```
end
```

```
out_prev = out;
```

```
in_prev = in;
```

```
end
```



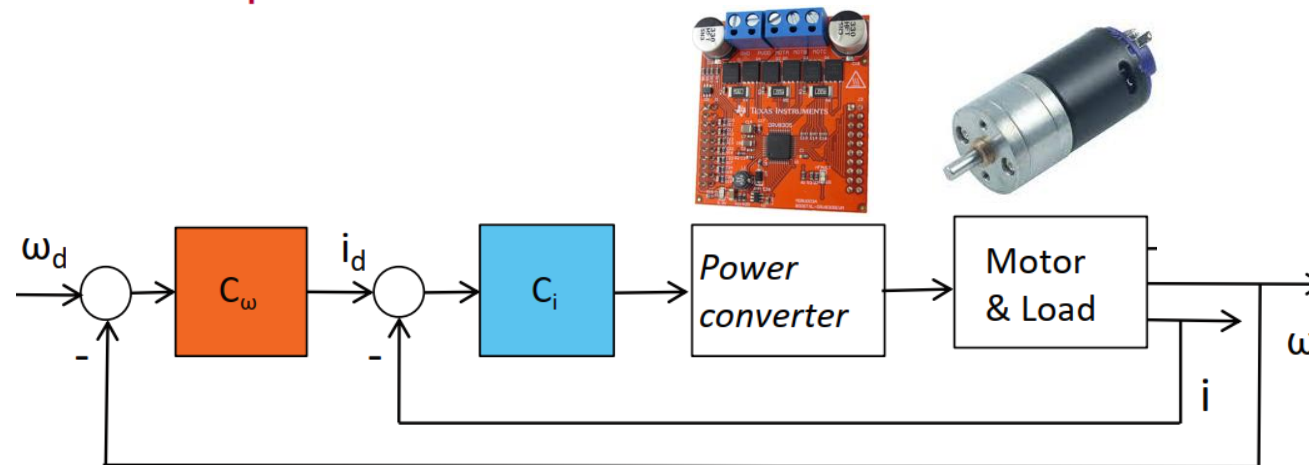
IMPORTANT: when we will use this matlab function in BaseModelForControl, we have to restore -1

From **LAS_03_Motion.pdf** (Prof. Melchiorri)

Cascade control of an electric motor

In “high performance” (high dynamics) applications, the control of an electric motor can be “naturally” implemented with a cascade structure, composed of:

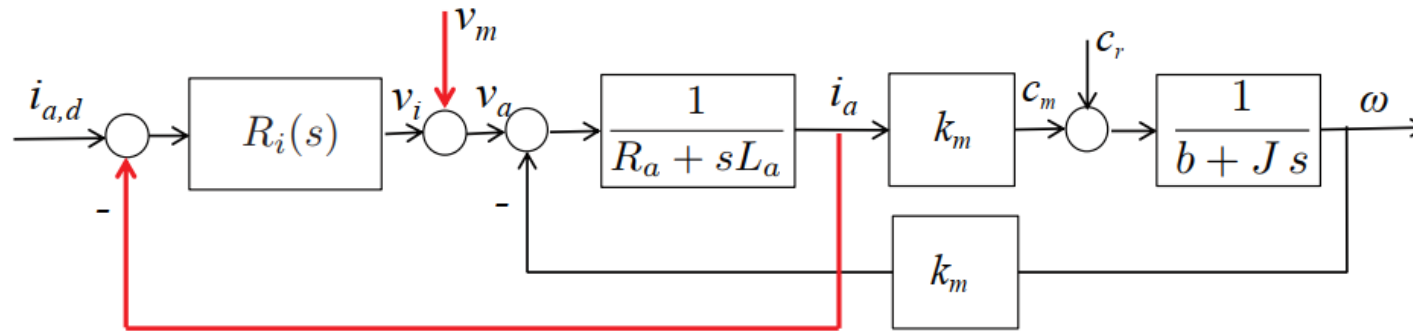
- current control loop, or *torque* control loop
- velocity control loop



Kp?
Ti?

- Watch Prof. Melchiorri Slide (Design of Motion Control System) from pag. 26

The classical block scheme for the current control loop is



By assigning a desired time constant τ_i for the current loop, it results

$$\frac{R_a T_{ii}}{K_{pi}} = \tau_i, \quad \rightarrow \quad K_{pi} = \frac{L_a}{\tau_i}$$

Therefore, the two parameters of the PI current controller are:

$$T_{ii} = \tau_a = \frac{L_a}{R_a}, \quad K_{pi} = \frac{L_a}{\tau_i}$$

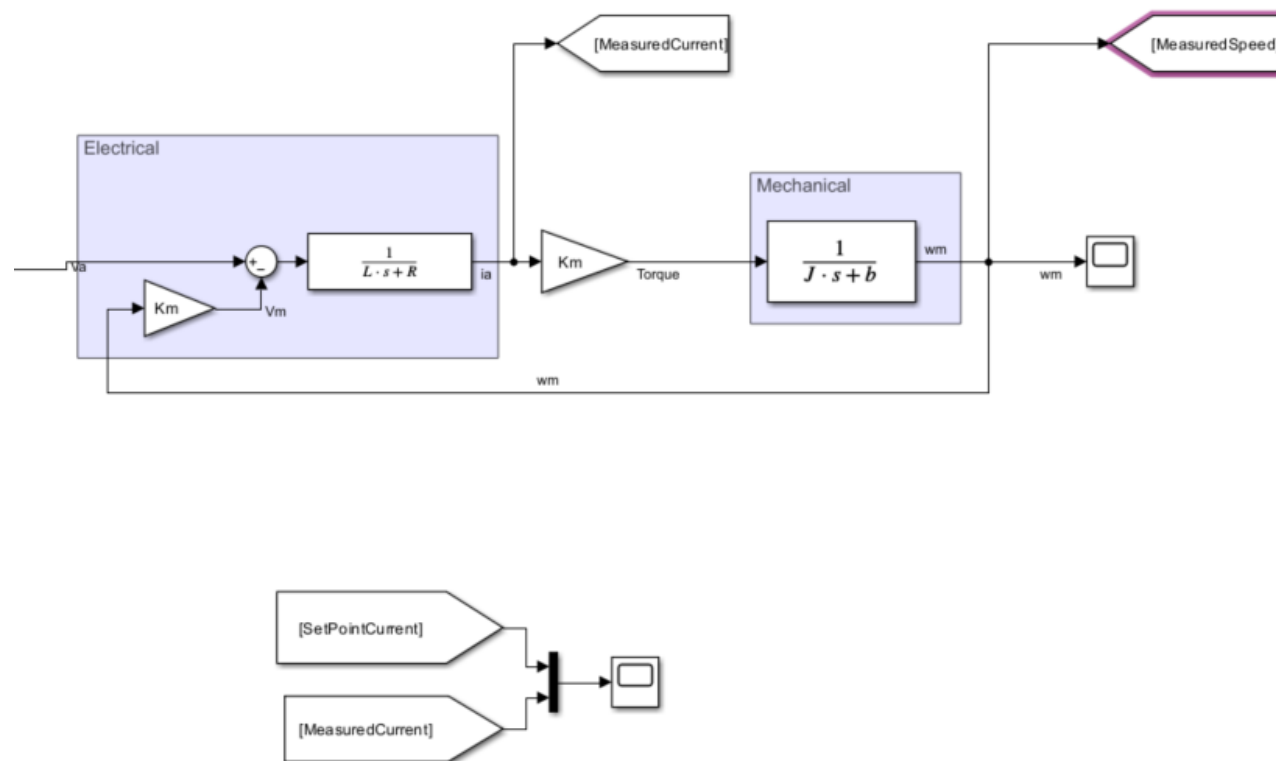
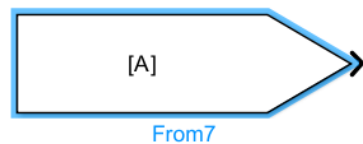
NOTE:

The mechanical dynamics of a DC motor is described by the function

$$G_m(s) = \frac{1}{b + Js} = \frac{1/b}{1 + \frac{J}{b}s} = \frac{1/b}{1 + \tau_m s}, \quad \tau_m = \frac{J}{b}$$

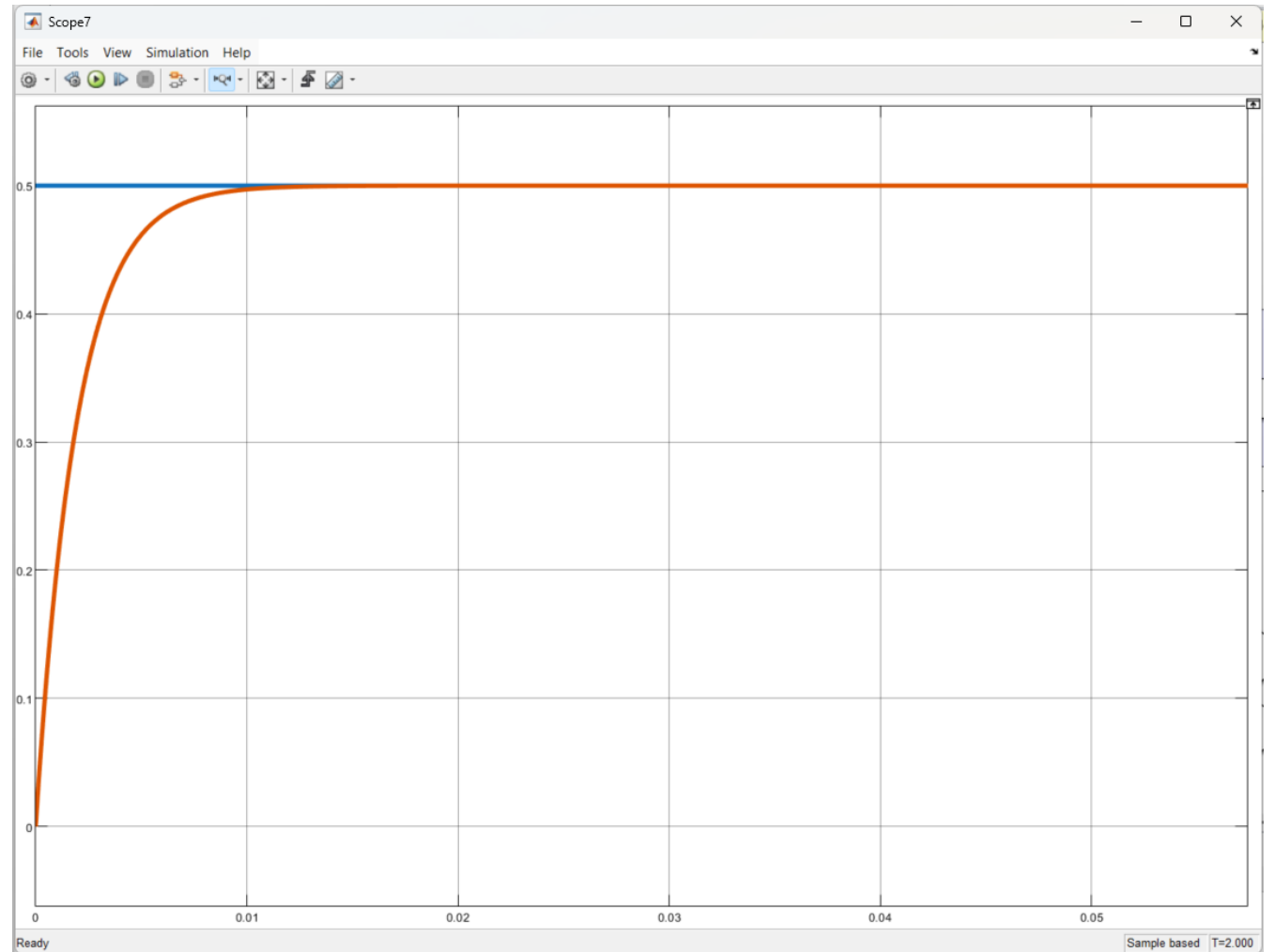
DC Motor datasheet

If you use Goto-From block you'll have a more readable scheme



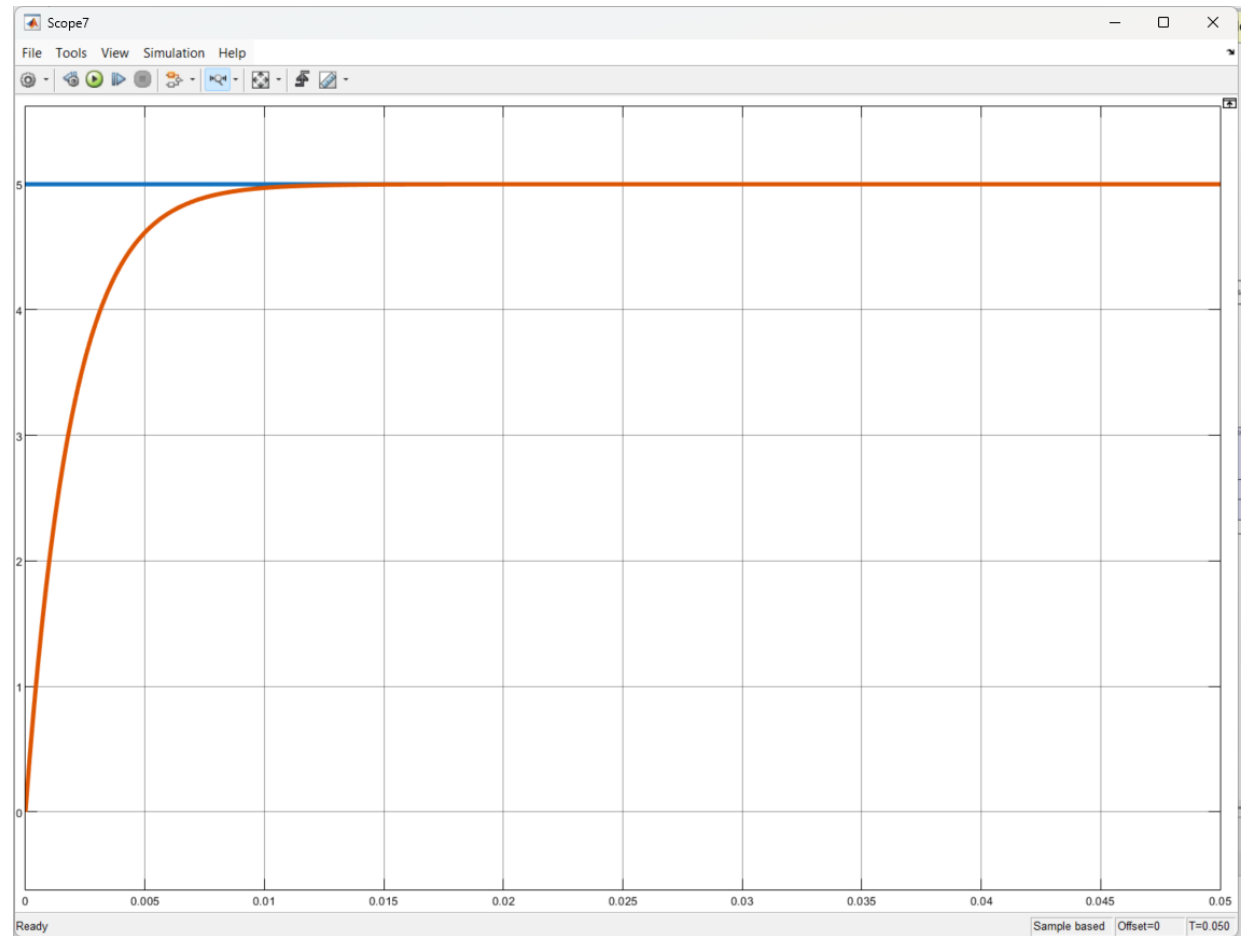
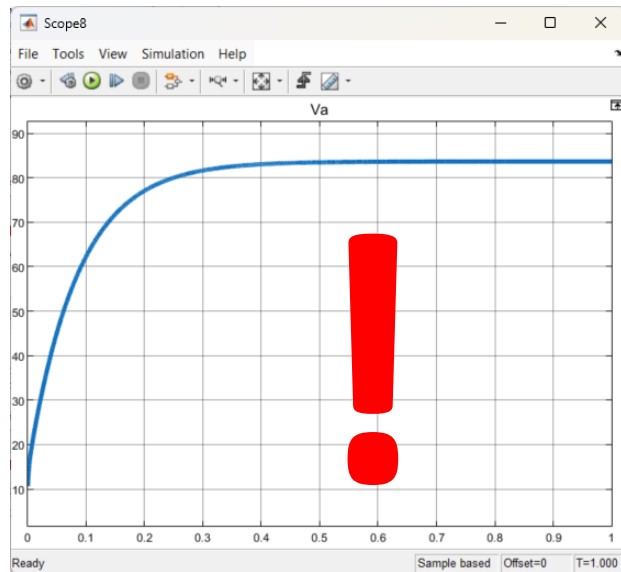
1 TEST:

- Apply a **0.5** Ampere SetPoint
- Watch SetPoint and measured current

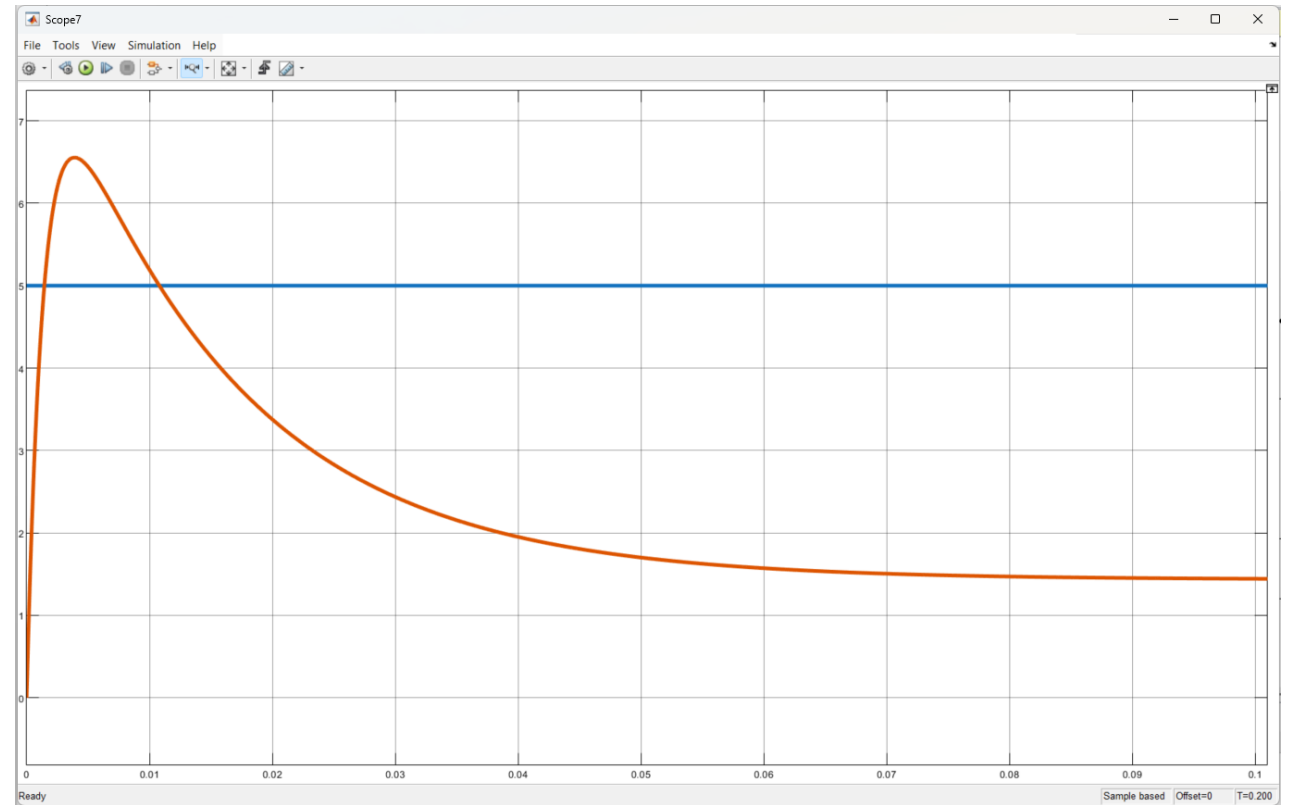
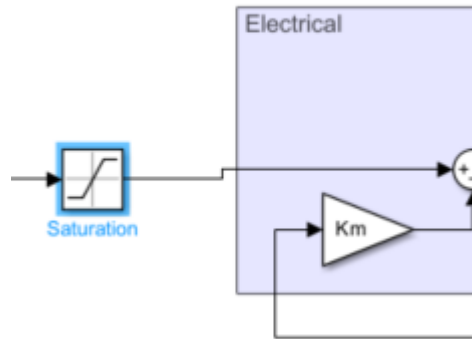


2 TEST:

- Apply a 5 Ampere SetPoint
- Watch SetPoint and measured current
- Check DC Motor Voltage

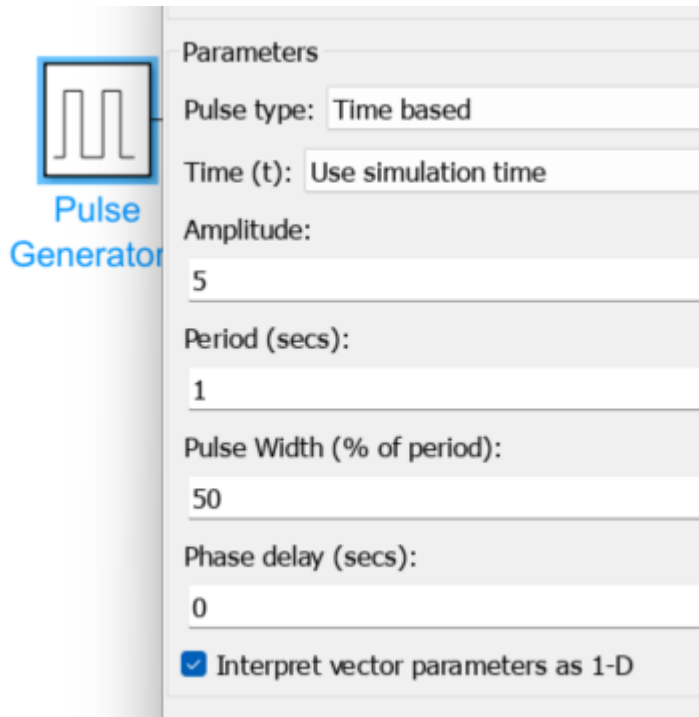


- Put a saturation +24/-24V
- Watch SetPoint and measured current

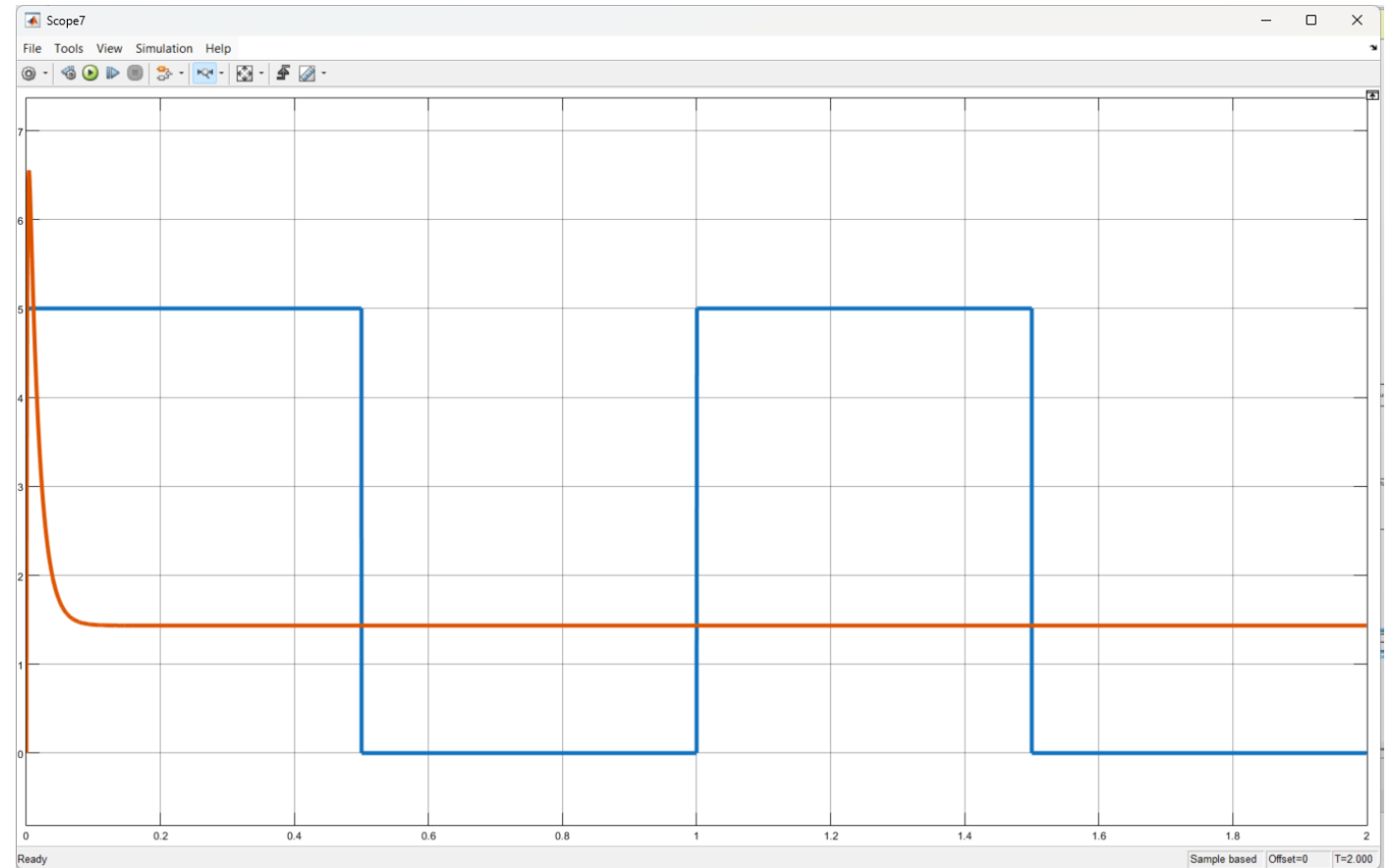


Let's have a chat about this....

- Remove Step and use Pulse Generator
- Watch SetPoint and measured current

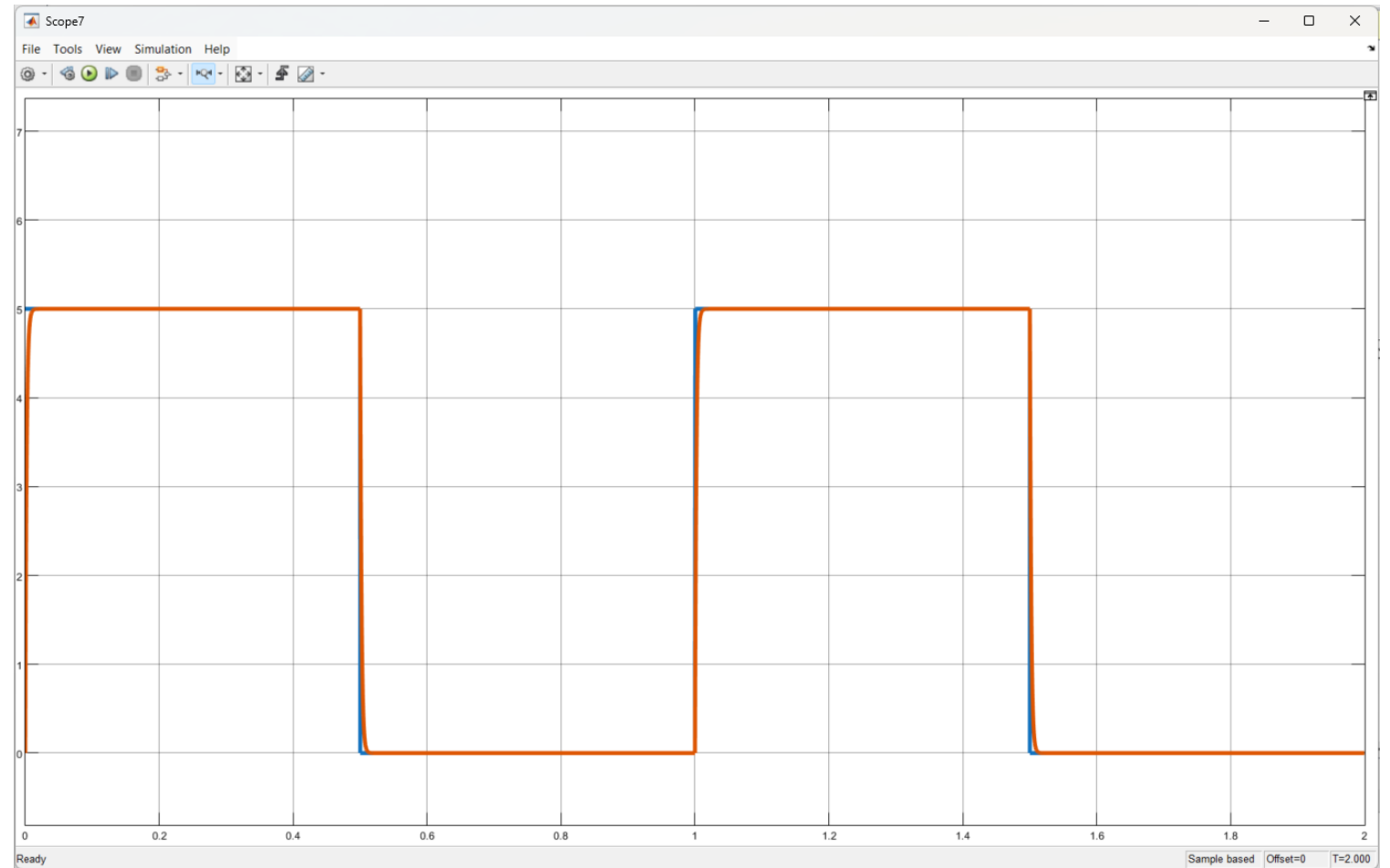


Let's have a chat about this....



- Remove Saturation
- Watch SetPoint and measured current

Let's have a chat about this....



SATURATION

- In Prof. Melchiorri slide you can find different anti windup technics, for the next time have a look and prepare one solution **HOMEWORK**
- Now we can quickly apply a fast improvement but not perfect.
 - After calculating the output we saturate the value

```
out = .....;

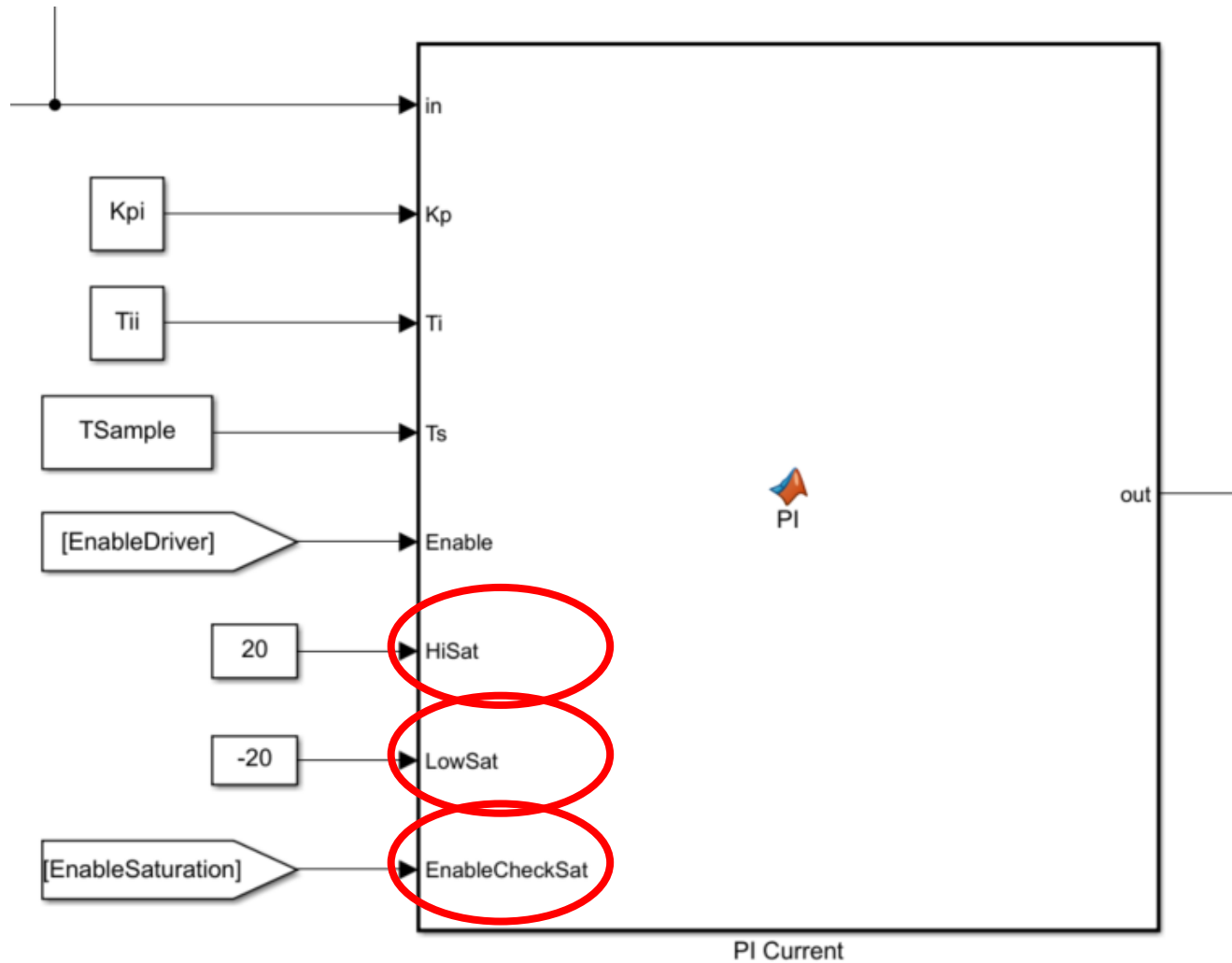
if(EnableCheckSat == 1)

    %check for saturation
    if out > HiSat
        out = HiSat;

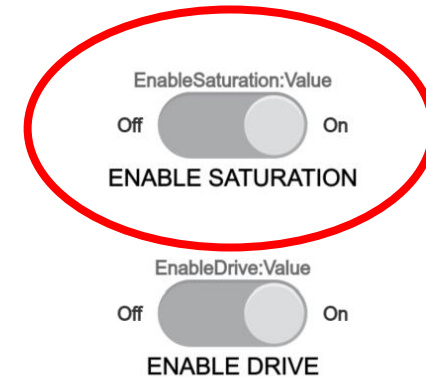
        elseif (out < LowSat)
            out = LowSat;

    end
end
```

Difference with c2d()

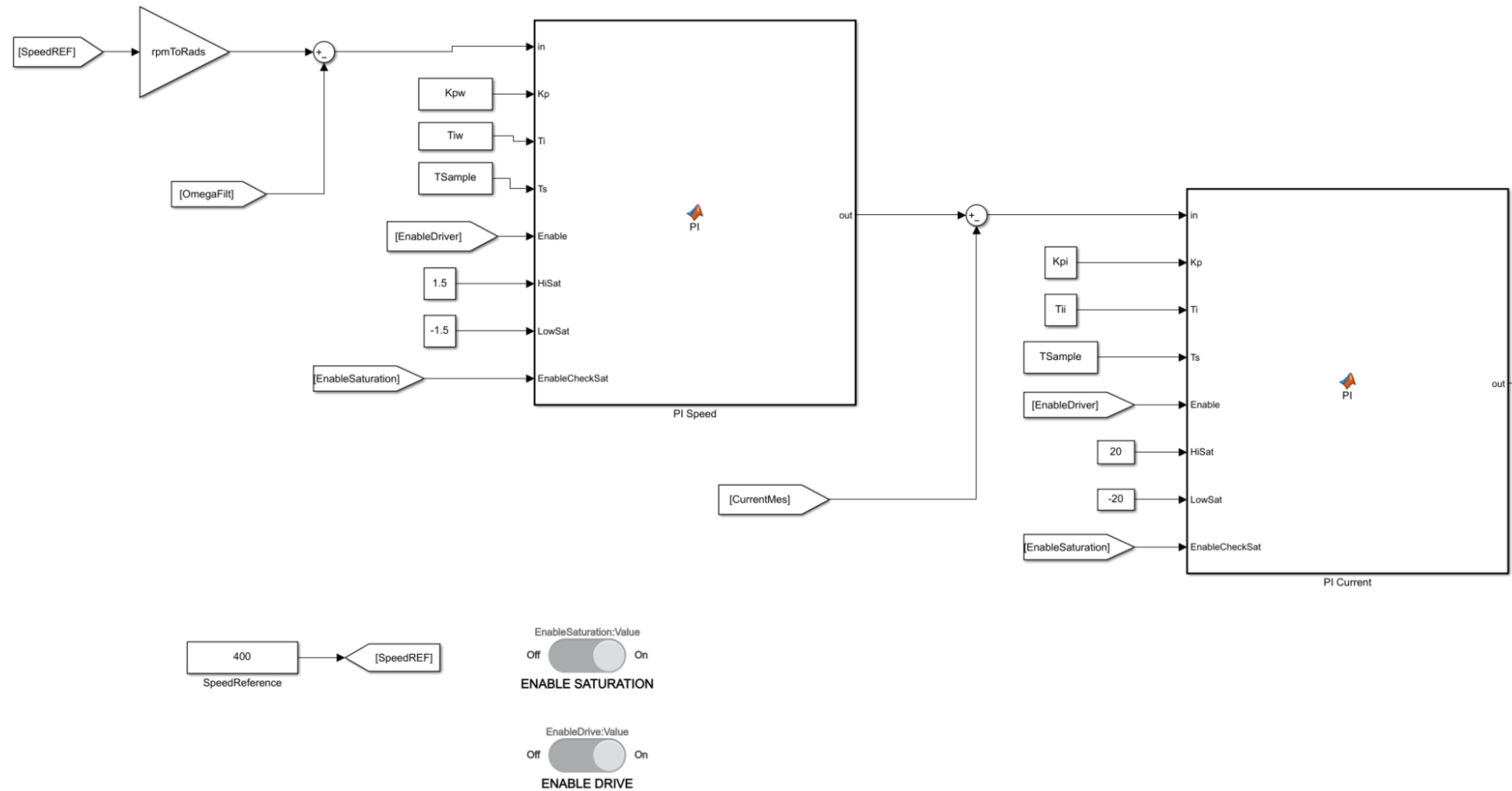


Add another Slider Switch



**Try what's happen now
with and without
EnableSaturation**

Now we can close also the speed loop:
(Method V2)



- pay attention to the unit of measurement of ω
 - What's about saturation value for Speed Loop?
-
-
-
-
-
-
-
-
-
-
- With this model you can also try to Implement P/Z Mapping